Missão 3 - SecureDocs

Resumo das implementações, escolha do algoritmo e entrega da chave

a) Implementações de DES, RC5 e AES

DES (des.py). O código cifra blocos de 8 bytes. A chave recebida também tem 8 bytes, mas apenas 56 dos seus 64 bits são efetivos. A implementação aplica a permutação inicial, deriva 16 subchaves por meio de PC-1, rotações e PC-2, e executa 16 rodadas Feistel. A função de cada rodada expande a metade direita, combina-a com a subchave por XOR, consulta as oito S-boxes e aplica a permutação P. Após a troca final das metades, aplica a permutação final. A decifragem reutiliza a estrutura com as subchaves em ordem inversa. Para sequências de bytes, cifrar e decifrar usam CBC com IV aleatório de 8 bytes, preenchimento PKCS#7 e formato IV || texto cifrado. testes_des.py cobre um vetor conhecido, a recuperação do texto e entradas inválidas.

RC5 (rc5.py). A classe fixa palavras de 32 bits, formando blocos de 64 bits, e usa 12 rodadas por padrão. A quantidade de rodadas pode ser informada ao construtor, enquanto o comprimento da chave é o número de bytes recebidos. A expansão da chave prepara a tabela de subchaves a partir das constantes P e Q. Cada rodada combina soma módulo 2^{32}, XOR e rotações cuja distância depende dos dados; a decifragem desfaz essas operações em ordem inversa. As rotinas de arquivo completam os dados em blocos de 8 bytes com preenchimento PKCS#7. Nessa rotina, cada bloco é cifrado separadamente, sem IV ou encadeamento: blocos iguais podem revelar padrões. teste_rc5.py inclui um vetor conhecido e uma ida e volta; teste_arquivo.py demonstra a recuperação de um arquivo. A chave de 16 bytes mostrada nos testes é apenas uma configuração do projeto.

AES (aes.py). A implementação própria opera sobre blocos de 16 bytes e aceita chaves de 16, 24 ou 32 bytes, correspondentes a AES-128, AES-192 e AES-256. Ela gera a S-box, expande a chave em subchaves de rodada e aplica SubBytes, ShiftRows, MixColumns e AddRoundKey, além das operações inversas na decifragem. Para dados com comprimento arbitrário, cifrar usa CBC, preenchimento PKCS#7 e um IV aleatório de 16 bytes, colocado antes do texto cifrado. testes_aes.py verifica vetores de bloco dos três tamanhos de chave e a ida e volta. experimento_arquivos.py gera uma chave aleatória de 32 bytes, cifra contrato.pdf por padrão, recupera o arquivo e compara o SHA-256 do original e do recuperado. Esse hash verifica a igualdade no teste local; ele não autentica o arquivo cifrado. O script grava a chave em hexadecimal, sem proteção, em *.key.

b) Algoritmo escolhido para o SecureDocs

O grupo escolheu AES e demonstrou AES-256-CBC com chave aleatória de 256 bits e IV aleatório. DES, com chave efetiva de 56 bits e padrão retirado, permanece como estudo histórico. RC5 permite diferentes parâmetros, mas sua rotina de arquivo neste projeto cifra blocos independentemente e expõe padrões. O AES possui padronização vigente e, no experimento, permite cifrar e recuperar o documento. A implementação atual de CBC fornece apenas a demonstração da confidencialidade e da reversibilidade: ela não autentica o texto cifrado nem garante a detecção de alterações. Um sistema real também precisaria proteger a

chave e acrescentar autenticação dos dados. Não apresentamos esta implementação didática como pronta para produção.

c) Como entregar a chave secreta ao destinatário?

A proposta é gerar uma chave AES aleatória para cada arquivo e obter a chave pública autenticada do destinatário. Antes de usá-la, o sistema precisa confirmar a identidade à qual a chave pública pertence, por exemplo por um certificado validado ou outro canal de confiança. Um mecanismo de encapsulamento ou transporte apropriado, como RSA-OAEP, protege a chave AES para essa chave pública. Enviam-se o documento cifrado e a chave AES protegida; somente quem controla a chave privada correspondente consegue recuperar a chave AES e decifrar o documento. A chave privada deve permanecer protegida pelo destinatário. O código atual não implementa esse fluxo: salva o segredo diretamente em *.key, que não pode ser entregue junto do documento sem proteção. A autenticação do documento cifrado também continua pendente.

Fontes técnicas

NIST FIPS 46-3 - DES: https://csrc.nist.gov/pubs/fips/46-3/final

Rivest - The RC5 Encryption Algorithm: https://people.csail.mit.edu/rivest/pubs/pubs/Riv94.pdf

NIST FIPS 197 - AES: https://csrc.nist.gov/pubs/fips/197/final

NIST SP 800-38A - CBC: https://csrc.nist.gov/pubs/sp/800/38/a/final

NIST SP 800-56B Rev. 2 - transporte de chaves: https://csrc.nist.gov/pubs/sp/800/56/b/r2/final